

System Test Plan (STP)

EthicLens: AI Bias Detection & Mitigation Workbench

April 27, 2026

Team Fair – Group 7

Vardh Jain • Sangeetha Mahendra • Preeti Das

System Test Plan (STP)

EthicLens: AI Bias Detection & Mitigation Workbench

REVISION DATE: May 3rd, 2026

The purpose of this System Test Plan (STP) is to establish a structured, repeatable framework for validating that the EthicLens AI Bias Detection and Mitigation Workbench meets all functional, non-functional, and performance requirements prior to production deployment. According to Ammann and Offutt (2016), a system test plan serves as the authoritative contract between the development team and stakeholders, ensuring that every specified behavior can be demonstrated to be correct, complete, and consistent. This STP defines the testing scope, methodology, roles, test scripts, and acceptance criteria that govern all verification and validation activities for EthicLens across the George Washington University SEAS graduate software engineering course project lifecycle.

The objectives of this System Test Plan (STP) are:

1. To verify that all functional requirements of EthicLens — including model ingestion, synthetic persona generation, disparate impact computation, mitigation recommendation, and governance sign-off — are correctly implemented and traceable to test evidence.
2. To validate that all non-functional requirements, including response time, concurrent user capacity, and system availability, are satisfied under realistic load conditions.
3. To ensure that each testing type (unit, function, integration, performance, and user acceptance) is executed in a systematic, documented, and reproducible manner, supporting both course deliverable requirements and real-world software quality assurance standards.

The scope of this System Test Plan covers all software components of the EthicLens three-tier architecture: the React.js front-end workbench interface, the Python 3.11+ back-end API and business logic layer (comprising the Synthetic Profile Generator, Disparate Impact Engine, and Mitigation Recommender modules), and the PostgreSQL data persistence layer. Testing activities described herein apply to the EthicLens system as designed for EquitaTech Systems and executed within the GWU SEAS Spring 2026 graduate software engineering project context. Out of scope are third-party ML framework internals (TensorFlow, PyTorch, Scikit-learn), cloud hosting infrastructure, and production security penetration testing.

Approval of the System Test Plan (STP) indicates an understanding of the purpose and content described in this deliverable. By signing this deliverable, each individual agrees with the content contained in this deliverable.

Approver Name	Title	Signature	Date
Vardh Jain	Developer / Tester	Vardh Jain	04/28/2026
Sangeetha Mahendra	Developer / Tester	Sangeetha Mahendra	04/28/2026
Preeti Das	Developer / Tester	Preeti Das	04/28/2026

Contents

1 Organization	5
1.01 Name of the Organization:	5
1.02 Industry of the Organization:	5
1.03 Mission Statement of the Organization:	5
1.04 Business Sponsor Role:	5
1.05 Technical Sponsor Role:	5
1.06 The Role of Software Development Team within the organization:	5
1.07 System Test Plan (STP) Approval Role:	5
2 Project Scope	6
2.01 Title of System	6
2.02 System/Project Scope	6
2.03 Project Importance	6
2.04 Background	6
2.05 Critical Success Factors	6
2.06 Key Performance Indicators (KPIs)	6
2.07 Planned Milestones for the Software Solution	7
2.08 Planned Milestones for the Class Project	7
2.09 Description of the System	7
2.10 Competing Systems	7
2.11 System Differentiation	7
3 System Testing	8
3.01 Research	8
4 Testing Approach and Method	10
4.01 Overall Approach	10
4.02 Testing Roles	10
4.03 Testing Definitions	11
5 Test Script Format	12
6 Unit Testing Test Script	13

7 Function Testing Test Script	15
8 Integration Testing Test Script	17
9 Performance Testing Test Script	19
10 User Acceptance Testing Test Script	21
11 Glossary	23
12 References	24
Appendix A Example Test Script	25

1 Organization

1.01 Name of the Organization:

EquitaTech Systems

1.02 Industry of the Organization:

AI Governance and Ethical Software Quality Assurance

1.03 Mission Statement of the Organization:

To empower organizations to build trust in AI by providing transparent, automated, and actionable bias detection and mitigation workflows.

1.04 Business Sponsor Role:

Chief AI Ethics Officer (CAIEO)

1.05 Technical Sponsor Role:

VP of Machine Learning Engineering

1.06 The Role of Software Development Team within the organization:

A specialized R&D unit responsible for architecting the core auditing engine and the interactive workbench user interface. The team operates under Scrum methodology, delivering incremental working software across four sprints.

1.07 System Test Plan (STP) Approval Role:

Head of AI Product Management

2 Project Scope

The purpose of the Project Scope is to specify the overall purpose of the system requirements that will govern the development and implementation of EthicLens.

2.01 Title of System (5 words describing the title of the system. Copy this section from the Project Scope).

EthicLens: AI Bias Mitigation Workbench

2.02 System/Project Scope

EthicLens provides a stress-test environment where data scientists and ML engineers subject AI models to rigorous fairness audits before deployment. The workbench ingests trained models, generates synthetic demographic personas, calculates standardized fairness metrics across protected attribute groups, and recommends prescriptive interventions to neutralize detected bias. All findings are captured in a structured Fairness Scorecard and routed through a formal governance sign-off portal before a model is cleared for production.

2.03 Project Importance

Organizations deploying automated decision-making systems face significant legal, reputational, and social risks when those systems exhibit algorithmic bias. With the enactment of the EU AI Act and increasing U.S. regulatory scrutiny of algorithmic fairness, companies require a structured framework for demonstrating fairness compliance. EthicLens prevents high-profile failures — such as biased recruiting or loan-approval models — by catching disparate impact before models reach production and by providing documented audit trails that satisfy regulatory reviewers.

2.04 Background

Traditional Software Quality Assurance focuses on functional correctness: does the system crash, does it return the right answer? EthicLens addresses a critical gap in the QA landscape: does the system treat all demographic groups equitably? Existing tools such as IBM AI Fairness 360 provide code libraries but require deep technical expertise and offer no collaborative workbench interface. Google's What-If Tool provides visualization but lacks automated mitigation logic. EthicLens fills this gap with a collaborative, three-module workbench accessible to both engineers and non-technical compliance officers, combining automated bias detection with prescriptive remediation in a single platform.

2.05 Critical Success Factors

- 2.01 Framework-agnostic integration:** EthicLens must ingest TensorFlow, PyTorch, and Scikit-learn models without requiring model refactoring.
- 2.02 Low barrier to entry for non-technical users:** Compliance Officers must be able to read and act on Fairness Scorecard reports without engineering assistance.
- 2.03 Mitigation efficacy without accuracy degradation:** recommended fixes must demonstrably reduce bias scores while preserving overall model predictive performance.
- 2.04 Key Performance Indicators (KPIs)**
- 2.05 Bias Variance Reduction:** percentage decrease in Disparate Impact scores between initial and post-mitigation audits, targeting at least 20 percentage points reduction per sprint cycle.
- 2.06 Audit Completion Time:** total hours saved by moving from manual bias testing to the automated EthicLens workflow, targeting a 60% reduction versus baseline manual process.
- 2.07 Mitigation Adoption Rate:** percentage of production AI models that pass through EthicLens before receiving deployment clearance, targeting 100% within six months of launch.

2.08 Planned Milestones for the Software Solution

1. Milestone 1: Delivery of the Synthetic Profile Generator — edge-case demographic persona creation with configurable attribute distributions.
2. Milestone 2: Finalization of the Disparate Impact Engine — standardized fairness metric calculations including 4/5ths Rule, Statistical Parity Difference, and Equalized Odds.
3. Milestone 3: Integration of the Mitigation Recommender — automated fix logic with ranked mitigation strategy output and model re-audit capability.
4. Milestone 4: Launch of the Governance Workflow module — formal audit trails, Fairness Scorecard PDF export, and executive sign-off portal.

2.09 Planned Milestones for the Class Project

5. Phase 1: Project Scope and Requirement Analysis — completed.
6. Phase 2: System Design Document (SDD) — completed.
7. Phase 3: Storyboard and UI Wireframes — completed.
8. Phase 4: System Test Plan (STP) — current deliverable.

2.10 Description of the System

EthicLens is a three-tier, browser-based AI bias auditing workbench designed for data scientists, ML engineers, and compliance professionals. The front-end layer is built in React.js, providing an interactive dashboard for model upload, audit configuration, metric visualization, and governance sign-off. The back-end layer is implemented in Python 3.11+ using a RESTful API architecture, and houses three core modules: the Synthetic Profile Generator, which creates thousands of synthetic demographic personas for stress-testing model decisions; the Disparate Impact Engine, which computes industry-standard fairness metrics (4/5ths Rule, Statistical

Parity Difference, Equalized Odds) across all protected attribute groups; and the Mitigation Recommender, which produces ranked, executable remediation strategies for flagged bias. Audit session data, fairness metrics, and mitigation logs are persisted in a PostgreSQL relational database organized around a five-entity data model with AuditSession as the central hub entity connected to UserAccount, ProtectedAttribute, FairnessMetric, and MitigationLog.

2.11 Competing Systems

IBM AI Fairness 360 (AIF360) is an open-source Python toolkit developed by IBM Research that provides a comprehensive suite of fairness metrics and bias mitigation algorithms. AIF360 supports over twenty fairness metrics and ten mitigation algorithms, making it one of the most technically comprehensive tools in the space. However, AIF360 operates exclusively as a code library with no graphical interface, no collaborative workflow, and no governance sign-off capability. Users must be proficient Python developers to configure and interpret results, effectively excluding compliance officers and product managers from the bias review process. EthicLens directly addresses these limitations by wrapping comparable metric computation inside an accessible workbench with a structured governance layer.

Google's What-If Tool (WIT) is a visual interface integrated into TensorBoard and the Google Cloud AI Platform that enables users to probe model behavior interactively and visualize performance across demographic subgroups. WIT is effective for exploratory analysis but lacks automated mitigation recommendations — it identifies that a problem exists but provides no actionable guidance on how to fix it. WIT is also tightly coupled to TensorFlow and Google Cloud infrastructure. EthicLens complements WIT's visualization strength with a framework-agnostic model ingestion layer, automated mitigation logic, and formal compliance workflows absent from WIT entirely.

2.12 System Differentiation

2.13 Integrated Mitigation Recommendations: Unlike AIF360 (code-only) and What-If Tool (visualization-only), EthicLens combines detection and prescriptive remediation in a single collaborative workbench, closing the loop from problem identification to documented fix.

2.14 Non-Technical Accessibility: The React.js front-end and Fairness Scorecard PDF output are explicitly designed for compliance officers and product managers, not just ML engineers, broadening the audience who can meaningfully participate in AI governance.

2.15 Framework-Agnostic, Governance-Ready: EthicLens ingests TensorFlow, PyTorch, and Scikit-learn models without modification, and provides a formal sign-off portal with audit trails, making it ready for regulatory compliance use cases that neither AIF360 nor the What-If Tool support.

3 System Testing

3.01 Research

System testing represents the most comprehensive phase of the software quality assurance lifecycle, verifying that a fully integrated system meets its specified requirements before it is released to end users. According to Ammann and Offutt (2016), system testing validates the behavior of the complete system as a whole, as opposed to unit testing, which validates individual components in isolation. The authors distinguish between verification — confirming that the system was built correctly according to specifications — and validation — confirming that the correct system was built to satisfy actual user needs. For EthicLens, both concerns are critical: the system must not only compute fairness metrics accurately (verification) but must also present those results in a way that enables compliance officers to act on them with confidence (validation).

The IEEE Standard for Software and System Test Documentation (IEEE Std 829-2008) provides the definitive framework for structuring test plans in professional software engineering contexts. The standard mandates that a System Test Plan document the test scope, test items, features to be tested, features not to be tested, approach, pass/fail criteria, suspension and resumption requirements, test deliverables, testing tasks, environmental needs, responsibilities, staffing and training needs, and schedule. While this STP is prepared in the context of the GWU SEAS graduate software engineering course, it adheres to the structural intent of IEEE Std 829-2008, ensuring that each test script is traceable to a specific requirement, includes defined preconditions, articulates discrete test steps, and specifies measurable expected outcomes. The standard's emphasis on traceability is particularly important for EthicLens, where bias detection results must be demonstrably reproducible for regulatory audit purposes.

Myers, Sandler, and Badgett (2011) argue in *The Art of Software Testing* that the most productive mindset for a tester is to assume that every system contains defects waiting to be found. This philosophy shapes the EthicLens testing strategy: rather than designing tests intended to confirm that the system works, the test scripts in this STP are designed to probe boundary conditions, unexpected inputs, and cross-module interactions where failures are most likely to emerge. For the Disparate Impact Engine, this means testing not only the happy-path scenario where a model exhibits clear bias, but also edge cases such as zero-variance attribute groups, numerically equal group selection rates (expected DI = 1.0), and models that produce predictions outside the $[0, 1]$ probability range. Myers et al. also caution against allowing developers to test their own code; in the EthicLens context, test execution roles are assigned across team members to ensure independent verification.

Bertolino (2007), in a landmark paper on software testing research challenges, observes that the testing of AI and machine learning systems introduces unique complications absent from traditional software: nondeterminism, data dependency, and the absence of a clear oracle. For EthicLens, this challenge manifests in the Disparate Impact Engine, where the “correct” fairness metric values depend on the input model, the demographic distribution of synthetic personas, and the protected attribute configuration chosen by the tester. The testing approach for EthicLens addresses this oracle problem by using calibrated reference datasets — pre-built

audit sessions with known bias characteristics — as golden standards against which system outputs can be mechanically compared, eliminating reliance on tester judgment for correctness determination.

Sommerville (2016) in *Software Engineering* (10th ed.) frames system testing within a broader verification and validation process that encompasses both static analysis (reviews, inspections) and dynamic testing (executing the software). For a complex, multi-module system like EthicLens, Sommerville recommends a layered test strategy proceeding from unit tests through integration tests to system tests and finally acceptance tests, each layer building on the confidence established by the prior one. This layered strategy is directly reflected in the EthicLens STP structure: unit tests in Section 6 establish confidence in individual metric functions; integration tests in Section 8 verify that the three modules communicate correctly; and User Acceptance Tests in Section 10 confirm that compliance officers and ML engineers can successfully execute their real-world workflows. The research reviewed here consistently supports this incremental, evidence-based approach to quality assurance, and EthicLens adopts it as its governing testing philosophy.

4 Testing Approach and Method

4.01 Overall Approach

EthicLens adopts a layered, risk-driven testing strategy aligned with the Scrum agile methodology used throughout the project. Testing is integrated into each sprint rather than deferred to a final phase, ensuring that defects are caught close to the point of introduction. The testing pyramid principle is applied: a broad base of fast-executing unit tests provides foundational confidence; a mid-layer of functional and integration tests verifies feature completeness and module interoperability; and a peak of performance and user acceptance tests confirms system readiness under realistic conditions. All test scripts are traceable to specific functional requirements (FR) or non-functional requirements (NFR) documented in the EthicLens System Design Document. A GitHub Actions continuous integration pipeline executes the full automated test suite on every pull request, preventing regressions from being merged into the main branch. Manual testing — including user acceptance and exploratory testing — is conducted by team members other than the developer who implemented the feature under test.

4.02 Testing Roles

The following roles are defined for EthicLens testing, with definitions grounded in professional software quality assurance literature (Myers, Sandler, & Badgett, 2011; Ammann & Offutt, 2016):

Test Manager: The Test Manager is responsible for coordinating all testing activities, maintaining the test plan, tracking defect resolution, and reporting test progress to stakeholders. According to Myers et al. (2011), the Test Manager must ensure that testing is treated as a first-class engineering activity rather than an afterthought. In the EthicLens project, this role is fulfilled by Vardh Jain, whose deep system knowledge informs test planning decisions.

Tester: The Tester executes test scripts, records actual results, identifies deviations from expected outcomes, and logs defects in the defect tracking system. Ammann and Offutt (2016) emphasize that effective testers must approach the system with adversarial intent. In EthicLens, Sangeetha Mahendra and Preeti Das serve as primary testers for modules they did not develop, ensuring independence between implementation and verification.

Project Manager: The Project Manager is responsible for ensuring that testing milestones align with the overall project schedule and that stakeholder communications regarding testing status are accurate and timely. Myers et al. (2011) note that the Project Manager must protect testing time from schedule compression, as deferred testing consistently produces higher post-release defect costs.

Lead Developer: The Lead Developer is responsible for producing testable code, maintaining unit test coverage targets, and resolving defects in priority order. According to Sommerville (2016), lead developers should treat test coverage data as a first-class engineering metric. In EthicLens, each module developer owns unit test coverage for their component.

User / Stakeholder: End users and stakeholders participate in User Acceptance Testing (UAT) to validate that the system satisfies real-world workflows and business needs. Bertolino (2007) notes that for AI systems, user acceptance is non-trivial because the “correctness” of a bias recommendation depends on domain expertise that only the stakeholder possesses. In EthicLens, the CAIEO persona serves as the primary UAT stakeholder.

4.03 Testing Definitions

The following testing types are applied within the EthicLens STP, each defined and cited in accordance with GWU library-accessible academic literature:

Unit Testing: Unit testing validates individual software components — functions, methods, or classes — in isolation from the rest of the system, with all dependencies replaced by mocks or stubs. Ammann and Offutt (2016) define unit testing as the foundational layer of the testing pyramid, providing the fastest feedback on the correctness of individual logic units. For EthicLens, unit tests are implemented using pytest for the Python back-end modules and Jest for the React.js front-end components.

Function Testing: Function testing verifies that each discrete user-facing feature of the system operates correctly end-to-end from the user’s perspective, as distinct from verifying internal implementation details. Myers et al. (2011) describe functional testing as black-box testing, where the tester interacts with the system through its public interface without knowledge of internal implementation.

Integration Testing: Integration testing validates the interactions and data flows between separate modules or services that have been individually unit-tested. According to Sommerville (2016), integration testing is where many of the most damaging defects are found, because module interface assumptions made by developers independently often prove incorrect when modules are combined.

Regression Testing: Regression testing re-executes a full or representative subset of existing test cases after code changes to verify that previously passing behaviors have not been broken by new modifications. Ammann and Offutt (2016) note that regression testing is most valuable when automated and integrated into a continuous integration pipeline.

Performance Testing: Performance testing evaluates system behavior under realistic or peak load conditions, measuring response times, throughput, resource utilization, and failure modes under stress. Myers et al. (2011) distinguish load testing (expected peak usage), stress testing (usage beyond expected limits), and soak testing (sustained load over time).

Acceptance Testing (UAT): User Acceptance Testing validates the system against real-world user workflows and business requirements, typically executed by or alongside actual stakeholders rather than by the development team. Bertolino (2007) emphasizes that for AI systems, UAT carries particular weight because automated tests cannot fully capture whether a bias recommendation is actually useful and interpretable by the intended user.

5 Test Script Format

A test script is a formal, structured document that specifies the exact sequence of actions a tester must perform, the environmental preconditions required, and the measurable expected outcomes against which actual results are compared. The selection of a rigorous test script format is not merely procedural — it is a core quality assurance decision with direct implications for the reproducibility, traceability, and auditability of testing evidence.

According to IEEE Std 829-2008 (IEEE Standard for Software and System Test Documentation), a conformant test case specification must include: a unique test case identifier, the items or features to be tested, the input specifications, the output specifications, environmental needs, any intercase dependencies, and pass/fail criteria. The standard further requires that test procedures specify each step in enough detail that a tester with appropriate but not exhaustive system knowledge can execute the procedure without ambiguity.

Myers, Sandler, and Badgett (2011) argue that test cases must be designed with the explicit goal of finding defects, not of demonstrating that the system works. Practically, this means that each EthicLens test script includes at least one boundary or edge-case step in addition to the nominal happy-path scenario, probing conditions where failures are most likely to emerge.

The format adopted for all EthicLens test scripts in this STP uses a structured table layout aligned with both IEEE Std 829-2008 and the testing literature. Each test script table includes: a unique Script ID (TS-[TYPE]-[NNN]); the test type; a plain-language description; requirements addressed (traced to FR or NFR codes from the EthicLens SDD); explicitly stated preconditions; numbered test steps paired with expected results; fields for actual results; and a pass/fail determination field. An illustrative example is included in Appendix A.

6 Unit Testing Test Script

Unit tests for EthicLens are implemented using pytest (Python back-end) and Jest (React.js front-end). The four scripts below address core calculation and data-handling functions in the Disparate Impact Engine and Synthetic Profile Generator modules.

Test Script: TS-UNIT-001 – Disparate Impact Ratio Calculation	
Test Type: Unit Testing	
Description: Verifies that the calculate_disparate_impact() function in the Disparate Impact Engine returns the correct 4/5ths Rule ratio given known group selection rates.	
Requirements Addressed: FR-003: System shall compute the Disparate Impact ratio for each protected attribute group.	
Preconditions: Python test environment configured; pytest installed; Disparate Impact Engine module imported; no database connection required (pure function test).	
Test Steps	Expected Results
Step 1	Import calculate_disparate_impact() from disparate_impact_engine.py.
Expected 1	Function is importable with no errors.
Step 2	Call calculate_disparate_impact(privileged_rate=0.80, unprivileged_rate=0.40).
Expected 2	Return value equals 0.50; system flags result as failing 4/5ths threshold ($0.5 < 0.8$).
Step 3	Call calculate_disparate_impact(privileged_rate=0.80, unprivileged_rate=0.80) to test equality edge case.
Expected 3	Return value equals 1.0; system marks result as PASS (no disparate impact).
Step 4	Call calculate_disparate_impact(privileged_rate=0.80, unprivileged_rate=0.0) to test zero-rate edge case.
Expected 4	Return value equals 0.0; system flags result as maximum disparity FAIL.
Step 5	Assert that a ValueError is raised when privileged_rate=0.0 (division by zero guard).
Expected 5	ValueError raised: 'Privileged group selection rate cannot be zero.'
Actual Result	Pass / Fail
Tester	
Date Executed	

Test Script: TS-UNIT-002 – Synthetic Profile Attribute Bounds Validation	
Test Type: Unit Testing	
Description: Verifies that the generate_profiles() function produces persona records with all attribute values strictly within defined configuration bounds.	
Requirements Addressed: FR-002: System shall generate synthetic demographic personas with configurable attribute distributions.	
Preconditions: Python test environment configured; pytest installed; Synthetic Profile Generator module imported; faker library available.	
Test Steps	Expected Results
Step 1	Define test config: age [18,65], gender ['Male','Female','Non-binary'], race ['White','Black','Hispanic','Asian'].
Expected 1	Function executes without errors.
Step 2	Call generate_profiles(n=1000, config=test_config) and store result.
Expected 2	profiles list is returned successfully.
Step 3	Assert len(profiles) == 1000.
Expected 3	Profile count is exactly 1000.
Step 4	Assert all profile['age'] values are integers in [18, 65] inclusive.
Expected 4	No age value falls outside [18, 65]; no non-integer ages present.
Step 5	Assert all profile['gender'] values are members of the configured options set.
Expected 5	No gender or race value outside the defined options sets.
Actual Result	Pass / Fail
Tester	
Date Executed	

Test Script: TS-UNIT-003 – Statistical Parity Difference Calculator	
Test Type: Unit Testing	
Description: Verifies that calculate_spd() returns the correct Statistical Parity Difference value and that the result is zero when two groups have equal positive prediction rates.	
Requirements Addressed: FR-003: System shall compute Statistical Parity Difference (SPD) as a secondary fairness metric.	

Preconditions: Python test environment configured; pytest installed; Disparate Impact Engine module imported.	
Test Steps	Expected Results
Step 1	Call calculate_spd(privileged_rate=0.70, unprivileged_rate=0.40) and capture result.
Expected 1	Function executes and returns a float.
Step 2	Assert return value equals -0.30 (0.40 – 0.70).
Expected 2	Return value is exactly –0.30 to four decimal places.
Step 3	Call calculate_spd(privileged_rate=0.60, unprivileged_rate=0.60) and assert result equals 0.0.
Expected 3	Return value is exactly 0.0; no floating-point imprecision beyond 1e-10.
Step 4	Assert SPD values in range (–0.1, 0.1) are labeled 'Acceptable' by classify_spd().
Expected 4	classify_spd(–0.08) returns 'Acceptable'.
Step 5	Assert SPD values ≤ -0.1 or ≥ 0.1 are labeled 'Flagged' by classify_spd().
Expected 5	classify_spd(–0.15) returns 'Flagged'; classify_spd(0.12) returns 'Flagged'.
Actual Result	Pass / Fail
Tester	
Date Executed	

Test Script: TS-UNIT-004 – Composite Bias Score Aggregation	
Test Type: Unit Testing	
Description: Verifies that compute_composite_bias_score() correctly aggregates multiple fairness metric values into a single weighted composite score using configured metric weights.	
Requirements Addressed: FR-003: System shall aggregate fairness metrics into a single Composite Bias Score per audit session.	
Preconditions: Python test environment; Disparate Impact Engine imported with default weights (DI: 0.40, SPD: 0.35, EO: 0.25).	
Test Steps	Expected Results
Step 1	Call compute_composite_bias_score(di=0.6, spd=-0.25, eo=0.15) with default weights.
Expected 1	Function returns a float between 0.0 and 1.0.

Step 2	Compute expected result manually: DI component = $0.6 \times 0.40 = 0.240$; SPD component = $(1 - -0.25) \times 0.35 = 0.2625$; EO component = $(1 - 0.15) \times 0.25 = 0.2125$. Expected composite = 0.7150.
Expected 2	Function returns 0.7150 (± 0.001 tolerance); calculation matches the three weighted components summed above.
Step 3	Assert return value equals expected result ± 0.001 .
Expected 3	Actual return value matches expected within tolerance.
Step 4	Assert that a score < 0.6 is classified as 'High Risk' by <code>classify_composite_score()</code> .
Expected 4	<code>classify_composite_score(0.55)</code> returns 'High Risk'.
Step 5	Assert that a score ≥ 0.8 is classified as 'Low Risk' by <code>classify_composite_score()</code> .
Expected 5	<code>classify_composite_score(0.85)</code> returns 'Low Risk'.
Actual Result	Pass / Fail
Tester	
Date Executed	

7 Function Testing Test Script

Function tests for EthicLens validate complete user-facing features from the front-end interface through the API layer. All function tests are executed in a staging environment with a seeded PostgreSQL test database.

Test Script: TS-FUNC-001 – Model Upload and Ingestion Feature	
Test Type: Function Testing	
Description: Validates the complete model upload feature: a user uploads a supported ML model file, the system validates the format, extracts metadata, and displays a success confirmation.	
Requirements Addressed: FR-001: System shall allow users to upload trained ML models in TensorFlow (.h5), PyTorch (.pt), or Scikit-learn (.pkl) format.	
Preconditions: Staging environment running; test user logged in; sample TensorFlow model file (test_model.h5, <100MB) available; PostgreSQL test database seeded.	
Test Steps	Expected Results
Step 1	Navigate to the EthicLens dashboard and click 'Upload Model'.
Expected 1	Upload dialog opens successfully; file is selectable.
Step 2	Select test_model.h5 from the local file system via the upload dialog.
Expected 2	File is accepted by the upload control without error.
Step 3	Click 'Submit' and wait up to 30 seconds for processing.
Expected 3	Success toast notification: 'Model uploaded successfully.'
Step 4	Attempt to upload an unsupported file type (test_model.csv) and observe system response.
Expected 4	Error message: 'Unsupported file type. Please upload a .h5, .pt, or .pkl file.'
Step 5	Verify that the uploaded model appears in the 'My Models' list with correct name and metadata.
Expected 5	Model entry visible in 'My Models' list showing framework, file size, and upload timestamp.
Actual Result	Pass / Fail
Tester	
Date Executed	

Test Script: TS-FUNC-002 – Audit Session Creation Feature	
Test Type: Function Testing	
Description: Validates that a user can configure and create a new audit session by specifying a model, selecting protected attributes, and configuring demographic group definitions.	
Requirements Addressed: FR-004: System shall allow users to create a new Audit Session linking an uploaded model, protected attributes, and demographic group configurations.	
Preconditions: Staging environment running; test_model.h5 already uploaded; user logged in as ML Engineer persona.	
Test Steps	Expected Results
Step 1	Click 'New Audit Session' from the dashboard.
Expected 1	New Audit Session wizard opens with model dropdown populated.
Step 2	Select test_model.h5 from the model dropdown.
Expected 2	test_model.h5 is selectable and displays metadata summary on selection.
Step 3	Add protected attribute 'Race' with groups: White, Black, Hispanic, Asian.
Expected 3	Race attribute added; groups listed in the configuration panel.
Step 4	Add protected attribute 'Gender' with groups: Male, Female.
Expected 4	Gender attribute added below Race; both shown in summary.
Step 5	Set the positive outcome label as 'Approved' and click 'Create Session'.
Expected 5	Session created; user redirected to Audit Session dashboard with session ID and attributes.
Actual Result	Pass / Fail
Tester	
Date Executed	

Test Script: TS-FUNC-003 – Fairness Scorecard PDF Generation	
Test Type: Function Testing	
Description: Validates that a completed audit session generates a downloadable Fairness Scorecard PDF containing all required metric values, attribute breakdowns, and composite bias scores.	

Requirements Addressed: FR-007: System shall generate a Fairness Scorecard PDF for completed audit sessions. FR-003: Scorecard shall include DI, SPD, and Equalized Odds values per protected group.	
Preconditions: Completed audit session (Session ID: TEST-001) with calculated metrics for Race and Gender attributes; user logged in.	
Test Steps	Expected Results
Step 1	Navigate to Session TEST-001 in the Audit Sessions list.
Expected 1	Session TEST-001 detail page loads with status 'Completed'.
Step 2	Click 'Generate Report' button.
Expected 2	'Generate Report' button is clickable and shows a spinner on click.
Step 3	Wait up to 15 seconds for PDF to generate.
Expected 3	Download prompt appears for 'EthicLens_Scorecard_TEST-001.pdf' within 15 seconds.
Step 4	Open the downloaded PDF and verify page 1 contains session header, model name, and date.
Expected 4	PDF page 1 correctly shows session ID, model name, audit date, and EquitaTech Systems letterhead.
Step 5	Verify the PDF contains a metrics table with DI, SPD, and EO values for each demographic group.
Expected 5	Metrics table shows DI, SPD, and EO for all demographic groups; Composite Bias Score displayed.
Actual Result	Pass / Fail
Tester	
Date Executed	

Test Script: TS-FUNC-004 – Mitigation Recommendation Display	
Test Type: Function Testing	
Description: Validates that the Mitigation Recommender surfaces ranked mitigation strategies for a flagged audit session and that each recommendation includes an explanation and estimated impact.	
Requirements Addressed: FR-005: System shall recommend ranked mitigation strategies where DI < 0.8. FR-006: Each recommendation shall include a plain-language explanation and estimated bias reduction impact.	
Preconditions: Audit session TEST-002 exists with DI = 0.62 for Race:Black (flagged); user logged in as ML Engineer.	
Test Steps	Expected Results

Step 1	Navigate to Session TEST-002 and locate the 'Mitigations' tab.
Expected 1	Session TEST-002 Mitigations tab visible with warning badge ('1 Flagged Group').
Step 2	Click the 'Mitigations' tab and wait for recommendations to load.
Expected 2	Recommendations panel loads within 5 seconds.
Step 3	Verify that at least two ranked recommendations are displayed.
Expected 3	Minimum two recommendations shown (Rank #1 and #2) with strategy name and estimated DI improvement.
Step 4	Click on the first recommendation to expand the detail view.
Expected 4	Detail panel opens with plain-language explanation, affected group, estimated post-mitigation DI.
Step 5	Click 'Apply Mitigation' on recommendation #1 and observe system response.
Expected 5	System shows 'Re-auditing...' status; session status updates to 'Mitigation in Progress'.
Actual Result	Pass / Fail
Tester	
Date Executed	

8 Integration Testing Test Script

Integration tests for EthicLens verify the data flows and interface contracts between the three core back-end modules and between the back-end API and the PostgreSQL persistence layer.

Test Script: TS-INT-001 – Synthetic Profile Generator to Disparate Impact Engine Pipeline	
Test Type: Integration Testing	
Description: Verifies that the Synthetic Profile Generator correctly produces a persona dataset that the Disparate Impact Engine can consume to compute fairness metrics with correct data types and expected bias results for a calibrated reference model.	
Requirements Addressed: FR-002: System shall generate synthetic personas. FR-003: System shall compute fairness metrics using generated personas.	
Preconditions: Integration test environment running; calibrated_bias_model.pkl (Scikit-learn model with known Race:Black DI ~0.55) loaded; 1,000-profile test persona set configured.	
Test Steps	Expected Results
Step 1	Call generate_profiles(n=1000, config=race_gender_config) from the Synthetic Profile Generator.
Expected 1	1,000 profile records returned with all required attribute fields.
Step 2	Pass the returned profile dataset and calibrated_bias_model.pkl to run_audit(model, profiles, protected_attrs=['Race','Gender']).
Expected 2	run_audit() executes without exception; log shows '1000/1000 profiles processed'.
Step 3	Assert that the Disparate Impact Engine processes all 1,000 profiles without error.
Expected 3	No processing errors; return code 200 from API endpoint.
Step 4	Assert the returned metrics dict contains keys for Race:Black DI/SPD and Gender:Female DI/SPD.
Expected 4	All four expected metric keys present in return dictionary.
Step 5	Assert 'Race:Black:DI' value is in range [0.45, 0.65] (expected ~0.55 per calibrated model).
Expected 5	Race:Black DI in [0.45, 0.65]; calibration validates pipeline consistency.
Actual Result	Pass / Fail
Tester	
Date Executed	

Test Script: TS-INT-002 – Disparate Impact Engine to Mitigation Recommender Handoff	
Test Type: Integration Testing	
Description: Verifies that bias flags produced by the Disparate Impact Engine are correctly passed to the Mitigation Recommender, which returns appropriate ranked mitigation strategies without data loss or format mismatch.	
Requirements Addressed: FR-003: Disparate Impact Engine produces bias flags. FR-005: Mitigation Recommender consumes bias flags to produce recommendations.	
Preconditions: Integration test environment running; audit_results pre-populated with Race:Black DI=0.62 (flagged) and Gender:Female DI=0.83 (passing).	
Test Steps	Expected Results
Step 1	Pass audit_results to get_recommendations(audit_results, model_metadata).
Expected 1	get_recommendations() returns dict with key 'Race:Black' containing a list of recommendations.
Step 2	Assert recommendations are returned only for the flagged Race:Black group (DI < 0.8 threshold).
Expected 2	'Race:Black' key contains at least two recommendation entries.
Step 3	Assert no recommendations are returned for Gender:Female (DI=0.83 > 0.8 threshold).
Expected 3	No key 'Gender:Female' present in recommendations output.
Step 4	Assert each recommendation dict contains keys: 'rank', 'strategy_name', 'description', 'estimated_di_improvement'.
Expected 4	Each recommendation dict contains all four required keys with non-null values.
Step 5	Assert recommendations are ordered by 'rank' ascending with no rank gaps.
Expected 5	Recommendations list is correctly ordered 1, 2, 3 with no skipped ranks.
Actual Result	Pass / Fail
Tester	
Date Executed	

Test Script: TS-INT-003 – Complete End-to-End Audit Pipeline
Test Type: Integration Testing

Description: Verifies the complete EthicLens audit pipeline from model ingestion through audit session creation, fairness metric calculation, mitigation recommendation, and persistence to the PostgreSQL AuditSession table.	
Requirements Addressed: FR-001 through FR-007: Full pipeline requirements. NFR-DB-001: All audit results shall be persisted to PostgreSQL within 5 seconds of completion.	
Preconditions: Full staging environment running (React.js, Python API, PostgreSQL); calibrated_bias_model.pkl available; test database clean.	
Test Steps	Expected Results
Step 1	POST /api/models/upload with calibrated_bias_model.pkl and assert 201 Created with model_id.
Expected 1	201 response returned; model_id is a valid UUID.
Step 2	POST /api/sessions/create with model_id, protected_attrs=['Race','Gender'], n_profiles=1000 and assert 201 Created with session_id.
Expected 2	201 response returned; session_id is a valid UUID; session in PostgreSQL with status='In Progress'.
Step 3	Poll GET /api/sessions/{session_id}/status until status='Completed' (max 60 seconds).
Expected 3	Session status transitions to 'Completed' within 60 seconds.
Step 4	GET /api/sessions/{session_id}/metrics and verify metrics JSON contains DI, SPD, and EO for Race and Gender groups.
Expected 4	Metrics JSON contains all expected keys; DI values match expected range for calibrated model.
Step 5	Query PostgreSQL AuditSession table and assert row exists with status='Completed' and non-null completed_at timestamp.
Expected 5	AuditSession row shows session_id, model_id, status='Completed', completed_at timestamp within 5 seconds.
Actual Result	Pass / Fail
Tester	
Date Executed	

9 Performance Testing Test Script

Performance tests for EthicLens use Apache JMeter for load generation and Locust for concurrent user simulation. All performance tests are executed against the staging environment, with baseline metrics captured before testing and compared against defined NFR thresholds.

Test Script: TS-PERF-001 – Large Model Audit Response Time	
Test Type: Performance Testing	
Description: Verifies that EthicLens completes a full bias audit for a large ML model (>500MB, >1M parameters) within the response time NFR of 90 seconds under single-user load.	
Requirements Addressed: NFR-PERF-001: System shall complete bias audit for models up to 500MB within 90 seconds under single-user load.	
Preconditions: Staging environment on reference hardware (4 vCPU, 16GB RAM); large_model.h5 (512MB TensorFlow variant) available; single test user account.	
Test Steps	Expected Results
Step 1	Start performance monitoring (CPU, RAM, response time) via the staging environment metrics dashboard.
Expected 1	Monitoring active; baseline CPU <5%, RAM <4GB before test.
Step 2	Upload large_model.h5 via POST /api/models/upload and record upload duration.
Expected 2	Upload completes within 30 seconds; 201 response received.
Step 3	Trigger full audit session via POST /api/sessions/create with n_profiles=1000.
Expected 3	Audit session created; processing begins immediately.
Step 4	Poll GET /api/sessions/{session_id}/status every 5 seconds and record total audit duration to status='Completed'.
Expected 4	Total audit duration from session creation to 'Completed' is ≤90 seconds.
Step 5	Record peak CPU utilization and RAM consumption during audit processing.
Expected 5	Peak CPU ≤80%; peak RAM ≤12GB; no out-of-memory errors in server logs.
Actual Result	Pass / Fail
Tester	
Date Executed	

Test Script: TS-PERF-002 – Concurrent User Load Test	
Test Type: Performance Testing	
Description: Verifies that EthicLens maintains functional behavior and acceptable response times when 50 concurrent users simultaneously execute audit sessions.	
Requirements Addressed: NFR-PERF-002: System shall support a minimum of 50 concurrent audit sessions without response time degradation beyond 20% of single-user baseline.	
Preconditions: Staging environment running; Locust load testing script configured with 50 concurrent users; each virtual user uses a pre-uploaded small_model.pkl (<10MB).	
Test Steps	Expected Results
Step 1	Record single-user audit completion baseline time.
Expected 1	Single-user baseline audit completion time recorded (target: ≤90 seconds for small model).
Step 2	Start Locust with 50 concurrent users, ramp-up over 30 seconds, sustained load for 5 minutes.
Expected 2	50 virtual users start; Locust dashboard shows all users active.
Step 3	Record median audit completion time, 95th percentile, and error rate during load test.
Expected 3	Median completion ≤108 seconds (120% baseline); 95th percentile ≤135 seconds; error rate <1%.
Step 4	Observe server logs for 500-series HTTP errors or database connection pool exhaustion.
Expected 4	No 500-series errors; no database connection pool exhaustion events.
Step 5	After load test, verify a fresh audit session completes successfully (system recovery check).
Expected 5	Post-load audit session completes within baseline +20%; system returns to stable state.
Actual Result	Pass / Fail
Tester	
Date Executed	

Test Script: TS-PERF-003 – Fairness Scorecard Report Generation at Scale
Test Type: Performance Testing
Description: Verifies that Fairness Scorecard PDF generation completes within the NFR time limit for audit sessions with the maximum number of protected attribute groups (10 attributes, 5 groups each).

Requirements Addressed: NFR-PERF-003: System shall generate Fairness Scorecard PDF for audit sessions with up to 10 protected attributes within 15 seconds.	
Preconditions: Completed audit session (PERF-TEST-003) with 10 protected attributes, 5 groups each (50 metric rows); PostgreSQL pre-populated with metric values.	
Test Steps	Expected Results
Step 1	Record the current server time as t_start.
Expected 1	t_start recorded; no existing report cached (cache cleared before test).
Step 2	Call POST /api/sessions/PERF-TEST-003/report and record HTTP response time.
Expected 2	API responds with 202 Accepted within 2 seconds; job_id returned.
Step 3	Wait for report generation; record t_end when download link is returned.
Expected 3	Download link returned within 15 seconds of POST request (t_end – t_start ≤15s).
Step 4	Download the generated PDF and verify page count and content completeness.
Expected 4	PDF contains ≥3 pages; all 50 demographic group metric rows present and correctly labeled.
Step 5	Repeat steps 1–4 five times and compute average and maximum PDF generation time.
Expected 5	Average PDF generation time ≤12 seconds across 5 runs; maximum single run ≤15 seconds.
Actual Result	Pass / Fail
Tester	
Date Executed	

10 User Acceptance Testing Test Script

User Acceptance Tests for EthicLens are executed by three stakeholder personas: the Compliance Officer (non-technical), the ML Engineer (technical), and the Governance Approver (executive). UAT is conducted in the staging environment using pre-populated audit sessions. Testers follow only the instructions provided in each script without consulting the development team during execution.

Test Script: TS-UAT-001 – Compliance Officer Bias Review Workflow	
Test Type: User Acceptance Testing	
Description: Validates that a non-technical Compliance Officer can locate a completed audit session, interpret the Fairness Scorecard, identify flagged demographic groups, and initiate an escalation — all without engineering assistance.	
Requirements Addressed: FR-007: Fairness Scorecard shall be interpretable by non-technical compliance personnel. FR-008: System shall provide an escalation workflow for flagged audit sessions.	
Preconditions: Compliance Officer persona account logged in; Session UAT-001 completed with Race:Black flagged (DI=0.62); no prior training beyond a 2-page user guide.	
Test Steps	Expected Results
Step 1	From the EthicLens dashboard, locate Session UAT-001 in the 'Recent Sessions' list.
Expected 1	Compliance Officer locates Session UAT-001 within 2 minutes without navigational assistance.
Step 2	Open the Fairness Scorecard and identify which demographic group is flagged.
Expected 2	Flagged group (Race:Black, DI=0.62) is visually distinct and identifiable within 1 minute.
Step 3	Without assistance, describe in writing what the flagged DI value of 0.62 means in plain language.
Expected 3	Written description correctly identifies that Black applicants are approved at 62% the rate of the favored group, below the 80% regulatory threshold.
Step 4	Download the Fairness Scorecard PDF and verify it opens correctly.
Expected 4	PDF downloads and opens without technical issues; content matches on-screen scorecard.
Step 5	Use the 'Escalate to Engineering' button to send a flagged session notification.
Expected 5	Escalation notification sent; ML Engineering team inbox shows notification with session link.
Actual Result	Pass / Fail

Tester	
Date Executed	

Test Script: TS-UAT-002 – ML Engineer Mitigation Application Workflow	
Test Type: User Acceptance Testing	
Description: Validates that an ML Engineer can review mitigation recommendations, apply the top-ranked recommendation, trigger a re-audit, and verify that the post-mitigation DI score improves above the 0.80 threshold.	
Requirements Addressed: FR-005: System shall recommend ranked mitigation strategies. FR-009: System shall support re-audit after mitigation to produce updated metric values.	
Preconditions: ML Engineer persona logged in; Session UAT-002 (Race:Black DI=0.65) has two ranked recommendations.	
Test Steps	Expected Results
Step 1	Navigate to Session UAT-002 and open the Mitigations tab.
Expected 1	Mitigations tab loads with two ranked recommendations; Rank #1 shows estimated DI improvement.
Step 2	Read Rank #1 recommendation and click 'Apply Mitigation'.
Expected 2	Mitigation applied; system displays 'Re-auditing with mitigation...' status indicator.
Step 3	Confirm the mitigation application dialog and wait for re-audit to complete (max 120 seconds).
Expected 3	Re-audit completes within 120 seconds; session status returns to 'Completed (Mitigated)'.
Step 4	Navigate to the updated Session UAT-002 metrics view and record the new Race:Black DI value.
Expected 4	New Race:Black DI value is ≥ 0.80 ; system displays green PASS indicator for this group.
Step 5	Download the updated Fairness Scorecard PDF and verify it reflects post-mitigation values.
Expected 5	Updated PDF shows pre- and post-mitigation DI values side-by-side; Race:Black status as 'PASS'.
Actual Result	Pass / Fail
Tester	
Date Executed	

Test Script: TS-UAT-003 – Governance Sign-Off Portal Workflow	
Test Type: User Acceptance Testing	
Description: Validates that a Governance Approver can review the final audit summary for a fully passing session, add a formal approval note, and digitally sign off the session, locking the record and triggering a deployment clearance notification.	
Requirements Addressed: FR-010: System shall provide a Governance Sign-Off portal for executive approval. FR-011: Signed-off sessions shall be locked and trigger a deployment clearance notification.	
Preconditions: Governance Approver account logged in with Sign-Off permissions; Session UAT-003 status='All Groups Passing'; deployment clearance notification template configured.	
Test Steps	Expected Results
Step 1	Navigate to the Governance portal and locate Session UAT-003 in the 'Awaiting Sign-Off' queue.
Expected 1	Session UAT-003 appears in 'Awaiting Sign-Off' queue; accessible within 2 minutes.
Step 2	Open the session summary and review the executive fairness summary.
Expected 2	Executive summary shows composite bias score, all-group status table (all green PASS), model name, and audit date.
Step 3	Type an approval note in the 'Approver Comments' field.
Expected 3	Approval note accepted (up to 500 characters); no input errors.
Step 4	Click 'Approve and Sign Off' and confirm the action in the dialog.
Expected 4	Sign-off recorded; status updates to 'Approved' within 3 seconds.
Step 5	Verify that Session UAT-003 status changes to 'Approved' and the session record is no longer editable.
Expected 5	Session shows 'Approved' with approver name and timestamp; ML Engineer sees session as read-only; deployment clearance email sent.
Actual Result	Pass / Fail
Tester	
Date Executed	

11 Glossary

Define any terms and acronyms required to properly interpret the requirements contained within this document.

System Test Plan (STP): A formal document that establishes the testing scope, methodology, roles, test scripts, and acceptance criteria for a software system. According to IEEE Std 829-2008, an STP serves as the authoritative contract between the development team and stakeholders, ensuring that every specified behavior can be demonstrated to be correct, complete, and consistent. Example: this document is the EthicLens STP for EquitaTech Systems' AI bias auditing workbench.

4/5ths Rule (80% Rule): A federal regulatory threshold established by the U.S. Equal Employment Opportunity Commission (EEOC Uniform Guidelines on Employee Selection Procedures, 1978) stating that an unprivileged group's selection rate must be at least 80% of the most-favored group's rate to avoid evidence of adverse impact.

Audit Session: The central EthicLens database entity linking an uploaded ML model, configured protected attributes, computed fairness metrics, and generated mitigation recommendations into a single governed workflow instance.

Composite Bias Score: A weighted aggregate of Disparate Impact, Statistical Parity Difference, and Equalized Odds values into a 0–1 score representing overall model fairness. Scores ≥ 0.80 are classified Low Risk.

Disparate Impact (DI): A fairness metric measuring the ratio of the positive outcome rate for an unprivileged group to the rate for the privileged group. A $DI < 0.80$ triggers a bias flag in EthicLens (Bertolino, 2007).

FR / NFR: Functional Requirement / Non-Functional Requirement: specifications documented in the EthicLens System Design Document governing system behaviors (FR) and quality attributes (NFR).

Regression Testing: Re-execution of the full test suite after code changes to verify that previously passing behaviors have not been broken (Sommerville, 2016).

SPD (Statistical Parity Difference): A fairness metric equal to the positive outcome rate of the unprivileged group minus the rate of the privileged group. Values in $(-0.1, 0.1)$ are considered acceptable.

UAT (User Acceptance Testing): Testing executed by or alongside end users and stakeholders to validate that the system meets real-world business needs, independent of technical correctness (Bertolino, 2007).

12 References

- 1. Ammann, P., & Offutt, J. (2016). Introduction to software testing (2nd ed.). Cambridge University Press.**
- 2. Bertolino, A. (2007). Software testing research: Achievements, challenges, dreams. In Proceedings of the International Conference on Software Engineering – Future of Software Engineering (FOSE '07) (pp. 85–103). IEEE.
<https://doi.org/10.1109/FOSE.2007.25>**
- 3. IEEE Computer Society. (2008). IEEE standard for software and system test documentation (IEEE Std 829-2008). IEEE.
<https://doi.org/10.1109/IEEESTD.2008.4578383>**
- 4. Myers, G. J., Sandler, C., & Badgett, T. (2011). The art of software testing (3rd ed.). Wiley.**
- 5. Sommerville, I. (2016). Software engineering (10th ed.). Pearson Education.**

13 Appendix A: Example Test Script

The following illustrates the standard test script format used throughout this STP. All test scripts in Sections 6–10 follow this template exactly.

Test Script: TS-EXAMPLE-000 – Example Test Script Format	
Test Type: Unit / Function / Integration / Performance / UAT	
Description: Brief description of what this test validates and why it matters for the EthicLens system. One to three sentences.	
Requirements Addressed: FR-XXX or NFR-XXX: Requirement ID and description from the EthicLens SDD that this test addresses.	
Preconditions: List all conditions that must be true before test execution begins (environment state, accounts, data, tools).	
Test Steps	Expected Results
Step 1	Step 1: Describe the first discrete action the tester must take.
Expected 1	Expected result corresponding to Step 1.
Step 2	Step 2: Describe the second action with no ambiguity.
Expected 2	Expected result corresponding to Step 2.
Step 3	Step 3: Continue with subsequent actions.
Expected 3	Expected result corresponding to Step 3.
Step 4	Step 4: Include boundary or edge-case steps in addition to the happy path.
Expected 4	Expected result for edge-case step — include specific numeric assertions where applicable.
Actual Result	Pass / Fail
Tester	
Date Executed	